JARVIS-OS

Especificación técnica

Asistente agéntico con control profundo de Linux,

basado en IronClaw, con HUD envolvente y voz bidireccional

Autor: Horelvis Castillo Mendoza
Versión: 0.2

Fecha: abril de 2026

1. Resumen ejecutivo

Este documento especifica Jarvis-OS, un asistente agéntico que actúa como capa de interacción primaria entre el usuario y un sistema operativo Linux. El objetivo es ofrecer control profundo del sistema (procesos, ficheros, servicios, escritorio, red) mediante voz bidireccional y una interfaz visual envolvente, manteniendo a la vez un modelo de seguridad estricto basado en capacidades, políticas declarativas y auditoría completa.

La arquitectura adopta IronClaw (NEAR AI, Rust, Apache-2.0/MIT) como núcleo cognitivo y de sandbox, y le añade una capa de integración Linux desarrollada como servidor MCP independiente. Esta separación permite reutilizar el modelo de seguridad de IronClaw para todo lo que es código no confiable, mientras que las acciones host-trusted (D-Bus, polkit, systemd, AT-SPI2) viven en un componente nativo bajo control directo del proyecto.

El backend de razonamiento es Claude (Anthropic) vía endpoint OpenAI-compatible, con un router de intención basado en un modelo más rápido y económico. La voz se gestiona con el stack ya consolidado del autor: Faster-Whisper (STT), XTTS-v2 (TTS) y Silero VAD. La interfaz visual es un HUD envolvente full-frame estilo Iron Man, con paleta cian eléctrico, anillo central de ondas continuas reactivas a la voz y telemetría real en los costados.

Cambios principales respecto a la v0.1:

- Se sustituye el HUD overlay minimalista por un HUD envolvente full-frame con lenguaje visual del género Jarvis cinematográfico. Cada elemento decorativo del género se mapea a información operativa real.
- El centro del HUD pasa de un orbe esférico a un anillo de ondas circulares continuas que reaccionan al espectro de voz en tiempo real.
- Se añade la decisión de autenticación combinada (biometría polkit + frase clave hablada) para acciones privileged y modo sysadmin.
- Se confirma el uso de notificaciones del sistema (libnotify) para resultados largos (listas de ficheros, salidas de comandos), evitando saturar el HUD ni la voz.

2. Objetivos y no-objetivos

2.1 Objetivos

- Ofrecer una experiencia de control de Linux por voz y texto en lenguaje natural, con un único agente persistente que vive como servicio del usuario.
- Habilitar control total del sistema operativo (lectura, mutación, acciones privilegiadas) bajo un modelo de seguridad de defensa en profundidad.
- Garantizar reversibilidad de las acciones destructivas mediante snapshots automáticos y un comando de rollback de primera clase.
- Mantener trazabilidad completa: toda intención del LLM, decisión de política y ejecución queda auditada en journald estructurado.
- Reutilizar IronClaw como núcleo agéntico, evitando reinventar el bucle de razonamiento, el sandbox WASM y la memoria persistente.
- Aceptar contribuciones futuras del upstream de IronClaw sin necesidad de mantener un fork divergente.
- Proveer una interfaz visual coherente con el imaginario Jarvis cinematográfico, donde cada elemento decorativo refleja un dato operativo real, no una animación falsa.

2.2 No-objetivos

- Construir un sistema operativo nuevo. Jarvis-OS corre sobre una distribución Linux estándar (Debian/Ubuntu/Fedora con systemd).
- Reemplazar el shell o el escritorio gráfico a nivel de proceso. El agente coexiste con la sesión gráfica y la shell del usuario, no las sustituye técnicamente.
- Funcionar offline en su totalidad. La inferencia depende de un endpoint LLM externo (Claude API por defecto); el modo local con Ollama está soportado pero no es el camino primario.
- Soportar Windows o macOS. La spec asume Linux con systemd, D-Bus y, preferentemente, Wayland.

3. Glosario

Término	Definición
Intención	Estructura tipada que el LLM emite para solicitar una acción. Nunca es código ejecutable directo.
Capacidad	Permiso explícito y limitado en el tiempo para invocar una clase de herramientas. Otorgado por el motor de políticas.
Tool host-trusted	Herramienta cuya ejecución requiere acceso real al SO (D-Bus, systemd, polkit). Vive fuera del sandbox WASM.
Tool sandboxed	Herramienta no confiable o generada dinámicamente. Vive dentro del sandbox WASM de IronClaw.
MCP	Model Context Protocol. Protocolo abierto para exponer herramientas a agentes LLM. IronClaw lo soporta nativamente.
Modo sysadmin	Estado temporal y explícito en el que se relajan políticas de confirmación a cambio de auditoría intensificada y autenticación reforzada.
HUD	Heads-Up Display. Capa visual envolvente que rodea el escritorio del usuario, semitransparente y siempre presente cuando el agente está activo.
Frase clave hablada	Frase preconfigurada por el usuario que se reconoce mediante speaker verification, usada como segundo factor para entrar en modo sysadmin.

4. Visión general de la arquitectura

Jarvis-OS se compone de cuatro procesos principales más persistencia compartida:

1. Voice Daemon (Python). Maneja wake-word, VAD, STT, TTS y envía las transcripciones al núcleo. Adicionalmente expone el espectro de voz en tiempo real al HUD vía socket Unix.
2. IronClaw (Rust, sin modificar). Núcleo agéntico con bucle de razonamiento, sandbox WASM, memoria híbrida y motor de routines.
3. Linux MCP Server (Rust). Servidor MCP independiente que expone capacidades del SO como tools host-trusted bajo políticas estrictas.
4. HUD Envolvente (Tauri/Rust). Interfaz visual full-frame estilo Iron Man con telemetría real, anillo de voz reactivo y notificaciones de confirmación.

La persistencia compartida vive en PostgreSQL con pgvector (memoria de IronClaw) y en journald (auditoría). Los snapshots de filesystem se gestionan en Btrfs o ZFS según la instalación.

4.1 Diagrama lógico

```
+-----------------------------------------------------------------+
|                      Voice Daemon (Python)                      |
|   wake-word -> VAD -> Whisper -> HTTP/SSE -> IronClaw            |
|         |                                                       |
|         +-> espectro voz (FFT) ------> HUD via socket Unix      |
|                                                                 |
|   chunks LLM <- IronClaw -> XTTS -> audio                       |
+----------------------------+------------------------------------+
                             | HTTP/SSE
+----------------------------v------------------------------------+
|                      IronClaw (Rust)                            |
|  Agent Loop <-> Tool Registry <-> WASM Sandbox                  |
|       |                |                                       |
|       |                +-> MCP client -> Linux MCP Server       |
|       v                                                         |
|  PostgreSQL + pgvector (memoria hibrida)                        |
+----------------------------+------------------------------------+
                             | MCP (stdio)
+----------------------------v------------------------------------+
|                  Linux MCP Server (Rust)                        |
|  Policy Engine (OPA) -> Confirm Bridge -> Tool Adapters         |
|         |                    |                  |               |
|         v                    v                  v               |
|   journald audit       D-Bus -> HUD       D-Bus, AT-SPI2,       |
|                                            polkit, systemd,     |
|                                            btrfs, NM, ...       |
+-----------------------------------------------------------------+
                             ^
                             | D-Bus events (confirmaciones)
+----------------------------+------------------------------------+
|                      HUD Envolvente (Tauri)                     |
|  Anillo voz <- socket Unix    Telemetria <- proc/sysfs          |
|  Capabilities <- IronClaw     Notif. <- libnotify (D-Bus)       |
+-----------------------------------------------------------------+
```

5. Componentes

5.1 Voice Daemon

Servicio de usuario (systemd --user) escrito en Python. Responsable del ciclo de voz completo y del feed de espectro al HUD.

- Wake-word: openWakeWord con modelo entrenado para la palabra clave "Jarvis". Latencia objetivo en idle: menos del 5% de un núcleo.
- VAD: Silero VAD, ya en uso en proyectos previos del autor.
- STT: Faster-Whisper distil-large-v3 para baja latencia; large-v3 disponible para sesiones de dictado largo.
- TTS: XTTS-v2 como motor por defecto para naturalidad. Piper como fallback rápido para respuestas cortas.

- Streaming bidireccional: el daemon empieza a sintetizar en cuanto IronClaw emite el primer chunk de texto, sin esperar la respuesta completa.
- Espectro al HUD: FFT con ventana de 1024 muestras a 16 kHz cada 64 ms. 32 magnitudes por bandas se exponen vía socket Unix al proceso del HUD.
- Speaker verification: módulo opcional para reconocer la frase clave hablada del usuario en modo sysadmin (Resemblyzer o ECAPA-TDNN).

5.2 IronClaw (núcleo agéntico)

Se utiliza IronClaw v0.16.x sin modificaciones del código fuente. La integración es exclusivamente vía configuración y vía consumo de su cliente MCP.

5.2.1 Configuración LLM

```
LLM_BACKEND=openai_compatible
LLM_BASE_URL=https://api.anthropic.com/v1
LLM_API_KEY=sk-ant-...
LLM_MODEL=claude-sonnet-4-7
```

El router de intención (paso previo) se configura como una llamada adicional a un modelo más económico (Claude Haiku) gestionada desde el voice daemon antes de invocar IronClaw.

5.2.2 Tools registradas

- Built-in: gestión de memoria, búsqueda, archivos del workspace.
- WASM dinámicas: skills procedimentales que el agente construye a demanda.
- MCP externas: tools de Linux expuestas por el Linux MCP Server.

> **Importante.** IronClaw soporta MCP nativamente. La integración con el SO no requiere modificar Cargo.toml ni recompilar IronClaw; basta con configurar el endpoint MCP en su archivo de configuración.

5.3 Linux MCP Server

Componente nuevo, propio del proyecto, escrito en Rust. Expone capacidades del SO como tools MCP. Es el corazón del control sobre Linux y donde reside la mayor parte del trabajo de seguridad.

5.3.1 Categorías de tools

Categoría	Ejemplos	Política por defecto
read	fs.list, fs.read, process.list, system.info, journal.query	ALLOW
mutate.user	fs.write (en $HOME), apps.launch, notification.send	ALLOW + notify
mutate.system	fs.write (/etc, /usr), service.reload	CONFIRM
network	web.fetch, mail.send, http.request	ALLOW (fetch) / CONFIRM (send)
destructive	fs.delete, process.kill, service.stop	CONFIRM + snapshot
privileged	package.install, service.enable, user.modify	CONFIRM + biometría + frase clave

5.3.2 Adaptadores

- D-Bus: zbus crate. Acceso a org.freedesktop.Notifications, org.freedesktop.systemd1, org.freedesktop.NetworkManager, MPRIS, login1.
- AT-SPI2: para introspección de interfaces gráficas (lectura de árboles UI). Vía atspi crate.
- Polkit: invocación de acciones privilegiadas con prompt biométrico. Acción polkit dedicada (org.jarvis.privileged) registrada en /usr/share/polkit-1/actions/.
- Systemd: control de unidades del usuario y del sistema vía D-Bus. No se ejecuta systemctl como subproceso.
- Btrfs/ZFS: snapshots tomados antes de cualquier acción destructive. Si el FS no soporta snapshots, fallback a copia en ~/.jarvis-trash/ con TTL.
- Ejecución de comandos arbitrarios: bubblewrap con perfil seccomp, namespaces y cgroups. No existe una tool genérica shell.exec; cada categoría está tipada.

5.4 HUD Envolvente

Aplicación Tauri (Rust + webview ligero) que renderiza una capa transparente full-frame encima del escritorio. Mantiene presencia continua mientras el agente está activo, con modos de fade dinámico para no saturar visualmente.

5.4.1 Lenguaje visual

Estilo Iron Man clásico: paleta cian eléctrico (#5DCBFF, #3AA8E8, #9FE6FF) sobre fondo transparente, tipografía mono fina (Consolas/JetBrains Mono) con tracking amplio, elementos geométricos en las esquinas, rejilla hexagonal de fondo muy sutil y scanlines horizontales casi imperceptibles.

5.4.2 Estructura del frame

- **Esquinas:** triángulos rellenos con líneas cortas que sugieren un marco. Identificadores técnicos en strips superior e inferior (modelo LLM en uso, latencia, hora, estado del snapshot, estado sysadmin).
- **Lateral izquierdo (SUBSYSTEMS):** barras horizontales con CPU, MEM, NET, GPU. Datos reales del sistema. Barra adicional de uso de contexto LLM (tokens consumidos / 200K).
- **Centro (anillo de voz):** tres anillos giratorios concéntricos con dasharrays distintos a velocidades distintas, con marcas cardinales en grados. En el interior, un anillo de ondas circulares continuas formado por seis a ocho capas de curvas Bézier que cierran el círculo.
- **Lateral derecho (CAPABILITIES):** lista de las seis categorías de tools con su política actual y su indicador de color. Refleja en tiempo real el estado del motor OPA.
- **Pie:** transcripción en vivo de lo que está dictando el usuario.

5.4.3 Anillo central de voz

El anillo es el corazón visual del HUD. Cada capa es una curva paramétrica cerrada con la fórmula:

```
r(theta, t) = R_base + suma( A_k * sin(n_k * theta + phi_k + omega_k * t) )
```

Las amplitudes A_k se modulan en tiempo real con bandas FFT del audio. Las graves alimentan los armónicos bajos (lóbulos amplios), los agudos alimentan los armónicos altos (rizado fino superpuesto). El resultado es que la voz no produce barras saltando sino que el anillo respira y se ondula completo. Las consonantes secas crean rizos rápidos; las vocales sostenidas hinchan el anillo en lóbulos amplios.

Comportamiento por estado del agente:

- **Reposo:** amplitudes al 5%, anillo casi liso. Solo respira muy lentamente con el ruido ambiente del micro.

- **Escuchando:** amplitudes enganchadas al espectro real del micrófono.
- **Hablando:** mismo widget, fuente cambia al stream del TTS. Jarvis te habla y ves su voz.
- **Procesando (LLM):** patrón pseudoaleatorio sintético. No hay voz pero el anillo se mueve, comunica trabajo en curso.
- **Sysadmin:** paleta a rojo/ámbar, amplitudes 20% extra, capa adicional con armónicos altos. Visualmente inconfundible.

5.4.4 Comportamiento del frame

- **Click-through siempre activo:** el HUD nunca roba foco; el escritorio debajo siempre es clickable. En X11 vía _NET_WM_STATE_BELOW; en Wayland vía protocolo layer-shell (wlr-layer-shell-unstable-v1).
- **Fade dinámico:** opacidad alta cuando hay actividad (escuchando, hablando, confirmando, modo sysadmin). Cae al 15-20% en idle prolongado.
- **Toggle:** hotkey global (configurable, por defecto Super+J) para ocultar/mostrar el HUD completo. "Jarvis, oculta" lo oculta por voz.
- **Multi-monitor:** el HUD vive solo en el monitor primario por defecto. Configurable a "sigue al cursor" o "todos los monitores".

5.4.5 Confirmaciones inline

Cuando una política devuelve CONFIRM, el HUD modifica su anillo central: el aro pasa a ámbar, las amplitudes se calman, y al lado del anillo emerge un panel breve con la acción propuesta, los argumentos clave y el impacto estimado. El usuario aprueba con voz, con tecla o ignora (timeout configurable, por defecto 30 segundos = rechazo).

Cuando la información no cabe en el panel inline (lista de 47 ficheros, output extenso), el HUD muestra un resumen y dispara una **notificación del sistema** (libnotify vía D-Bus) con los detalles, que el usuario puede consultar sin perder el foco de lo que estaba haciendo. Esto evita inflar el HUD y aprovecha la infraestructura nativa del escritorio.

6. Modelo de seguridad

Jarvis-OS implementa cinco capas de defensa entre la intención del LLM y la ejecución real en el SO. El principio rector es: el LLM nunca ejecuta, el LLM propone.

6.1 Las cinco capas

Capa	Función	Tecnología
1. Validación sintáctica	Tipos, rangos, paths canónicos, anti-injection de shell	serde + validators (Rust)
2. Motor de políticas	Decisión ALLOW / CONFIRM / DENY por tool, args, hora, contexto	OPA (Rego) embebido
3. Confirmación humana	HUD muestra acción + impacto; aprobación por voz, tecla, biometría o frase clave según severidad	D-Bus + Polkit + Speaker verification
4. Sandbox de ejecución	Aislamiento de comandos arbitrarios y código generado	bubblewrap + seccomp + cgroups + WASM

Capa	Función	Tecnología
5. Auditoría y rollback	Registro estructurado, snapshots, detección de anomalías	journald + Btrfs/ZFS

6.2 Defensas contra prompt injection

- Marcado de contenido externo (emails, PDFs, web) como untrusted en el system prompt. Toda instrucción que aparezca en contenido marcado debe tratarse como datos.
- Patrón de doble LLM en acciones destructive y privileged: una segunda llamada al LLM económico verifica la coherencia de la acción con la petición original del usuario antes de ejecutarse.
- Allowlist obligatoria de dominios para network. Dominios nuevos requieren confirmación explícita y se pueden añadir al allowlist desde el HUD.
- Tokens de capacidad con scope: cada tarea recibe un token que limita las tools y los recursos accesibles. Las tools verifican el token; un intento fuera de scope falla cerrado.
- Detección de anomalías: contadores deslizantes por categoría. Frecuencia inusual o ráfagas atípicas pausan al agente y notifican al usuario.

6.3 Modo sysadmin (autenticación combinada)

Estado temporal activado explícitamente por el usuario. La activación requiere autenticación de dos factores combinados:

- **Factor 1 — biometría (algo que eres):** huella o FIDO2 vía polkit. La acción polkit org.jarvis.sysadmin.activate exige autenticación cada vez, sin caché.
- **Factor 2 — frase clave hablada (algo que sabes + cómo suenas):** el usuario configura una frase única durante el onboard inicial. El voice daemon usa speaker verification para confirmar que la frase es pronunciada por el usuario registrado, no reproducida por un altavoz.

Durante el modo sysadmin:

- Se relajan algunas políticas CONFIRM hacia ALLOW para reducir fricción durante mantenimiento.
- La auditoría se intensifica: logging verbose y un snapshot adicional al activar y al desactivar el modo.
- Expira automáticamente tras un timeout configurable (por defecto 5 minutos).
- El HUD pasa a paleta roja completa con un contador regresivo siempre visible.
- El anillo de voz central queda en rojo con núcleo brillante para que sea imposible olvidarse.

Decisión de diseño. La autenticación combinada (biometría + frase clave hablada) protege contra dos vectores reales: alguien que tenga acceso físico al sensor biométrico mientras el usuario duerme, y alguien que pueda reproducir una grabación con la voz del usuario. Ambos factores juntos cierran ese ataque.

7. Flujos de ejecución

7.1 Flujo nominal — petición de lectura

5. Usuario dice: "Jarvis, ¿cuántos PDFs tengo en la carpeta de Melilla?"

6. Voice Daemon: wake-word → VAD → STT → texto.
7. Router de intención (Haiku) clasifica como query.
8. IronClaw recibe el texto, planifica, decide invocar fs.list.
9. Linux MCP Server valida (capa 1), política ALLOW (capa 2), ejecuta y audita (capa 5).
10. Resultado vuelve a IronClaw, que sintetiza la respuesta.
11. Voice Daemon recibe tokens en streaming y los envía a XTTS-v2.
12. Audio se reproduce mientras el modelo aún genera. El anillo del HUD se modula con la voz de Jarvis.

7.2 Flujo con confirmación — acción destructive

13. Usuario: "Jarvis, borra los logs antiguos del proyecto puerto."
14. IronClaw planifica fs.delete con paths concretos.
15. Linux MCP Server: política devuelve CONFIRM.
16. Confirm Bridge envía evento al HUD vía D-Bus. El anillo central pasa a ámbar.
17. Panel inline en HUD muestra: descripción, número de ficheros, espacio a liberar. Notificación libnotify con la lista detallada.
18. Jarvis pregunta por voz: "Voy a borrar 47 ficheros, 312 megas. ¿Confirmas?"
19. Usuario aprueba (voz, tecla o gesto).
20. Linux MCP Server toma snapshot Btrfs y ejecuta el borrado.
21. Auditoría completa en journald con hash del snapshot para rollback futuro.

7.3 Flujo privileged y activación de sysadmin

Para acciones privileged y para la activación del modo sysadmin, el flujo añade dos pasos de autenticación encadenados:

22. El agente (o el usuario) solicita la acción privileged.
23. Linux MCP Server invoca polkit con la acción org.jarvis.privileged. Polkit muestra prompt biométrico (huella o FIDO2).
24. Si la biometría es válida, el HUD pasa a estado "awaiting passphrase" y Jarvis pide la frase clave por voz.
25. El voice daemon graba la frase, la valida contra el modelo de speaker verification del usuario y la transcribe para comparar con la frase configurada.
26. Si ambos factores son válidos, la acción se ejecuta dentro del Linux MCP Server, que ya corre con capabilities Linux específicas (CAP_NET_ADMIN, CAP_SYS_ADMIN según necesidad), no como root general.
27. Auditoría completa: ambos factores, hashes, timestamps y detalles de la acción quedan en journald.

8. Persistencia y observabilidad

8.1 Memoria del agente

La memoria es responsabilidad de IronClaw. Tres niveles:

- Trabajo: contexto de la sesión actual, en memoria del proceso. El uso de tokens se refleja en la barra de contexto del HUD lateral izquierdo.
- Episódica: PostgreSQL, búsqueda full-text + vector con Reciprocal Rank Fusion (capacidad nativa de IronClaw).
- Procedimental: WASM tools dinámicas que IronClaw construye al detectar patrones repetidos.

8.2 Auditoría

Toda intención ejecutada o rechazada queda en journald con campos estructurados:

```
JARVIS_TOOL=fs.delete
JARVIS_USER_INTENT="borra los logs antiguos del proyecto puerto"
JARVIS_POLICY_DECISION=CONFIRM
JARVIS_USER_APPROVAL=approved_voice
JARVIS_AUTH_FACTORS=biometric+passphrase
JARVIS_SNAPSHOT_ID=btrfs:home_2026_04_28_142133
JARVIS_RESULT_HASH=sha256:e3b0c44...
```

La consulta se realiza con journalctl filtrando por estos campos. Se proporciona la herramienta jarvis audit (CLI) para consultas comunes, y un widget opcional en el HUD para mostrar las últimas N acciones.

8.3 Rollback

El comando jarvis undo lista las últimas N acciones reversibles y permite restaurar el snapshot asociado. La operación de undo es a su vez una acción destructive (sustituye estado actual) y, por tanto, requiere confirmación.

9. Plan de implementación por fases

Fase	Entregable	Criterio de éxito
F1 — Fundación	IronClaw instalado y configurado con Claude vía OpenAI-compatible. Voice daemon conectado por HTTP/SSE.	Conversación por voz con razonamiento y memoria, sin tools de SO ni HUD todavía.
F2 — Linux MCP read-only	Servidor MCP en Rust con tools de categoría read (fs, process, system).	El agente puede listar, leer, buscar, sin modificar nada.
F3 — HUD básico	Frame envolvente Tauri con anillo de voz reactivo, telemetría real, capabilities. Sin confirmaciones todavía.	El HUD se ve, no roba foco, refleja voz e información operativa real.
F4 — Mutaciones de usuario	Tools mutate.user con notificaciones libnotify.	Mover, copiar, abrir apps, escribir en $HOME funciona con feedback visible.
F5 — Política y confirmación	OPA embebido. Confirmaciones inline en el HUD. Polkit para privileged.	Acciones CONFIRM y privileged requieren aprobación correctamente.
F6 — Sandbox + reversibilidad	bubblewrap para comandos arbitrarios. Snapshots Btrfs y jarvis undo.	Acciones destructive son siempre reversibles. Tests de fuga del sandbox.

Fase	Entregable	Criterio de éxito
F7 — Sysadmin con doble factor	Modo sysadmin con biometría polkit + speaker verification + frase clave.	Activación requiere ambos factores; audit lo registra.
F8 — Anti prompt-injection	Marcado untrusted, doble LLM, allowlist de dominios.	Suite de pruebas de inyección documentada y bloqueada.
F9 — Skills procedimentales	Aprendizaje de patrones, propuesta automática de routines.	El agente propone una routine tras detectar patrón ≥ 3 veces.
F10 — Multimodalidad	Captura de pantalla vía portal XDG, integración con vision de Claude.	El agente puede responder a "qué pone en la ventana de la izquierda".

10. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Prompt injection desde contenido externo	Alto — ejecución no autorizada	Marcado untrusted, doble LLM, capability tokens, anomaly detection
Coste de la API LLM en uso intensivo	Medio — fricción económica	Router con Haiku, caché de respuestas idempotentes, modo local Ollama opcional
Wayland restringe automatización GUI y always-on-top	Medio — pérdida de funcionalidad	Preferir D-Bus y AT-SPI2. Layer-shell para HUD en wlroots. GNOME Mutter como segundo objetivo
IronClaw upstream introduce breaking changes	Bajo — coste de mantenimiento	No fork; versión taggeada. Las tools de Linux viven fuera de IronClaw
Latencia de voz percibida	Medio — UX degradada	Streaming TTS desde primer token, Piper como fallback rápido
Privacidad de datos sensibles enviados a la API	Alto — fuga de información	Modo local-only por sesión, redacción de PII antes de enviar, allowlist de directorios
HUD envolvente percibido como intrusivo	Medio — abandono del usuario	Click-through siempre, fade dinámico, hotkey de toggle, modo "solo voz" configurable
Falsificación de la frase clave hablada	Alto — bypass de sysadmin	Speaker verification (no solo ASR). Anti-replay con desafío aleatorio en activaciones críticas

11. Decisiones abiertas

- **Filesystem para snapshots:** Btrfs ofrece mejor integración nativa en Debian/Ubuntu; ZFS es más maduro pero requiere DKMS. Decisión inicial: Btrfs por defecto, ZFS soportado opcionalmente.
- **Compositor Wayland prioritario:** wlroots (Sway, Hyprland) ofrece soporte completo de layer-shell para el HUD; GNOME Mutter no implementa el protocolo. Decisión: prototipo en wlroots primero, GNOME segundo, X11 como fallback.

- **Modelo de speaker verification:** Resemblyzer (más simple, suficiente) vs ECAPA-TDNN (más robusto, más pesado). Decisión depende de F7.
- **Router de intención:** Haiku externa o un clasificador local (DistilBERT fine-tuneado). La decisión depende del coste real medido en F1.
- **Wake-word:** openWakeWord vs Porcupine. openWakeWord es libre y entrenable; Porcupine tiene mejor calidad pero licencia restrictiva. Decisión inicial: openWakeWord.
- **Densidad del anillo de voz:** entre 6 y 12 capas. Más capas = más "humo", más coste de render. Calibrar visualmente en F3.

12. Apéndices

12.1 Referencias técnicas

- IronClaw — github.com/nearai/ironclaw — licencia Apache-2.0 / MIT.
- Model Context Protocol — modelcontextprotocol.io
- Open Policy Agent — openpolicyagent.org
- XDG Desktop Portal — flatpak.github.io/xdg-desktop-portal
- AT-SPI2 — gitlab.gnome.org/GNOME/at-spi2-core
- zbus (Rust D-Bus) — docs.rs/zbus
- wlr-layer-shell-unstable-v1 — gitlab.freedesktop.org/wlroots
- Resemblyzer — github.com/resemble-ai/Resemblyzer

12.2 Estructura de directorios propuesta

```
~/.jarvis/
├── config.toml             # voice daemon, HUD, hotkeys, monitor
├── policies/               # políticas Rego del Linux MCP Server
│   ├── default.rego
│   ├── destructive.rego
│   └── privileged.rego
├── allowlists/
│   └── network.toml        # dominios aprobados para network tools
├── snapshots/              # índice de snapshots para jarvis undo
├── auth/
│   ├── speaker.npy         # embedding del usuario para verification
│   └── passphrase.hash     # hash de la frase clave configurada
├── hud/
│   └── theme.toml          # paleta, opacidades, posición, monitor
└── logs/                   # mirror local de auditoría

~/.ironclaw/                # gestionado por IronClaw — no tocar
```

12.3 Hotkeys globales

Atajo	Acción
Super+J	Mostrar/ocultar HUD envolvente

Atajo	Acción
Super+Shift+J	Activar modo de captura de pantalla para multimodalidad
Super+Alt+J	Solicitar activación de modo sysadmin (inicia flujo de doble factor)
Esc (con HUD activo)	Cancelar la confirmación o petición en curso